Latest updates: https://dl.acm.org/doi/10.1145/3688351.3689164

RESEARCH-ARTICLE

TwinPilots: A New Computing Paradigm for GPU-CPU Parallel LLM Inference

CHENGYE YU, Chinese University of Hong Kong, Hong Kong, Hong Kong

TIANYU WANG, Shenzhen University, Shenzhen, Guangdong, China

ZILI SHAO, Chinese University of Hong Kong, Hong Kong, Hong Kong

LINJIE ZHU, Sangfor Technologies Inc., Shenzhen, Guangdong, China

XU ZHOU, Sangfor Technologies Inc., Shenzhen, Guangdong, China

SONG JIANG, The University of Texas at Arlington, Arlington, TX, United States

Open Access Support provided by:

The University of Texas at Arlington

Sangfor Technologies Inc.

Shenzhen University

Chinese University of Hong Kong

PDF Download
3688351.3689164.pdf
16 March 2026
Total Citations: 16
Total Downloads: 2548

Published: 16 September 2024

Citation in BibTeX format

SYSTOR '24: The 17th ACM International Systems and Storage Conference
September 23 - 24, 2024
Virtual, Israel

Conference Sponsors:
SIGOPS

SYSTOR '24: Proceedings of the 17th ACM International Systems and Storage Conference (September 2024)
https://doi.org/10.1145/3688351.3689164
ISBN: 9798400711817

TwinPilots: A New Computing Paradigm for GPU-CPU Parallel LLM Inference

Chengye Yu
Chinese University of Hong Kong
Hong Kong, China
cyyu23@cse.cuhk.edu.hk

Tianyu Wang ✉
Shenzhen University
Shenzhen, China
tywang@szu.edu.cn

Zili Shao
Chinese University of Hong Kong
Hong Kong, China
shao@cse.cuhk.edu.hk

Linjie Zhu
Sangfor Technologies Inc
Shenzhen, China
zhulinjie@sangfor.com.cn

Xu Zhou
Sangfor Technologies Inc
Shenzhen, China
zhouxu@sangfor.com.cn

Song Jiang
University of Texas at Arlington
Arlington, USA
song.jiang@uta.edu

ABSTRACT

When trained Large Language Models (LLMs) become available, it is desirable to carry out LLM inferences at the user end with limited resources. A common belief on LLM inference is that GPU is essentially the only meaningful processor as almost all computation is tensor multiplication that GPU excels in. However, this belief and its practice are challenged by the fact that GPU has insufficient memory and runs at a much slower speed due to constantly waiting for data to be loaded from the CPU memory via a slow PCIe bus. This makes the CPU a processor with meaningful computing power that can be leveraged to accelerate the inference.

In this paper, we propose a new computing paradigm to accommodate the twin computing engines (GPU and CPU) and the hierarchical memory architecture (GPU and CPU memory) in an asymmetric multiprocessing framework named *TwinPilots*. This framework enables an effective scheduling that balances the speeds of the CPU and the GPU. Our proposed scheduling strategy maximizes CPU and GPU utilization and fully hides the data loading time behind the CPU compute time. We have implemented TwinPilots in PyTorch and evaluated it on several language models (OPT and Llama) with various model sizes (13B, 30B, and 33B) on a server with an NVIDIA A10G GPU with 24GB memory and an AMD EPYC 7R32 CPU with 256GB memory. Experiment results show up to 3.39× speedup on inference throughput and 72.1% data transfer reduction compared with SOTA schemes.

This work is licensed under a Creative Commons Attribution International 4.0 License.
SYSTOR '24, September 23–24, 2024, Virtual, Israel
© 2024 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1181-7/24/09.
https://doi.org/10.1145/3688351.3689164

CCS CONCEPTS

• **Computing methodologies** → *Cooperation and coordination.*

KEYWORDS

Heterogeneous computing, LLM inference acceleration

1 INTRODUCTION

Transformer-based [29] self-learning Large Language Models (LLMs) are changing the world with their powerful and versatile capabilities to perform various tasks, such as answering questions, summarizing articles, and translating languages [21, 31, 33]. To be more effective, the models become ever larger with hundreds of thousands, millions, billions, or even trillions of parameters. Example models include OpenAI's GPT [21], Google's PaLM [5], Meta's Llama [27], and DeepMind's LaMDA [26]. These models are usually pretrained at large and powerful GPU clusters for weeks before they are available for deployment in various application settings for inference services [4, 6, 8, 13, 14, 16, 23].

Compared with model training, model inference has its distinct characteristics. First, it runs in more diverse system settings (rather than only at resource-rich data centers). It may be integrated into an edge computing environment or run in mobile devices where high-end GPUs are not available. Second, it is more likely to be used in an interactive manner serving a few users who concurrently ask questions. In this use scenario, response latency (in terms of output tokens per second for individual users) is more important than the system's throughput. Third, its computation is much less than that used for the training, which makes its deployment in a lower-end GPU feasible in terms of computing power. However, the parameters produced by the training and other working data sets can be too large and thus easily exceed the GPU memory size.

As an example of the memory constraint, inference on the Llama-33B [27], a popular LLM model, requires around

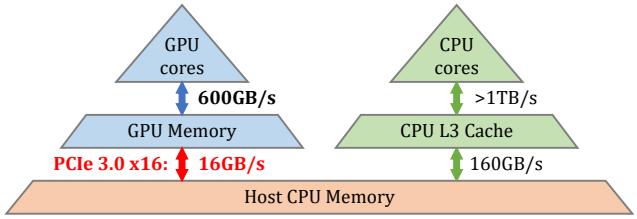


Figure 1: The GPU-CPU Architecture.

60GB of memory to fully hold its weight parameters represented in FP16 format, which exceeds the memory capacity of mainstream GPUs (e.g., the 24GB of NVIDIA A10G [17] or the 40GB of NVIDIA A100 [18]). If we consider the variable-length KV cache tensors, the memory shortage issue is even more serious. In the meantime, CPU memory can be of hundreds of Gigabytes. Consequently, a significant portion of the inference's working set has to reside in the CPU memory and be loaded into the GPU memory via the PCIe I/O bus. Since the bus is much slower than the GPU's memory bus (16GB/s vs. 600GB/s, as shown in Figure 1), which makes the GPU run at the I/O bus speed (a fraction of its full speed), GPU's computation power is under-utilized. This GPU memory bottleneck seriously keeps LLM model inference from being deployed in diverse environments with desirable user experiences.

A major approach to alleviating the bottleneck is to exploit access locality so that data loaded into the GPU can be reused as many times as possible before they have to leave the memory for space to accommodate the next batch of data from the CPU memory. Unfortunately, such reuse opportunity barely exists within an inference session, including the prompt processing and its output token generation, because a large number of transformer layers (typically 40 or more) are involved in a phase to generate an output token, and each of the layers keeps its own set of tensors. The only opportunity for improving the locality is to process the same layer of different sessions together before moving to the next layer so that the weight parameters of the layer shared by the sessions can be reused. This is the zigzag computation schedule proposed in the FlexGen inference design [24]. While this method improves inference throughput, individual sessions' response time can be seriously compromised. Accordingly, this optimization design is not applicable to interactive inference, which is a common application scenario. There are works that propose to overlap the data loading time with the GPU compute time such as PipeSwitch [3] and FlexGen [24]. They try to overlap the GPU computation of the current layer with the next layer's data loading and the previous layer's KV cache storing. This approach is not effective for the LLM inference as the GPU compute time is significantly dwarfed by the I/O time and the latter can hardly be hidden. By statically partitioning the computation of different layers between the CPU and GPU, Llama.cpp [7] introduces the CPU's computing power into the inference. However, its performance degrades quickly with larger batches and longer sequences. Larger batches would impose an excessive computing burden on the CPU, and longer sequences require a larger reserved working set in the GPU memory for an increased KV cache size for each layer, which inevitably reduces GPU utilization and increases the CPU load. Other works like DeepPlan [12] attempt to reduce I/O by transferring required parameters on demand at the page granularity without loading the whole layer. However, the computation of transformer-based LLMs is dominated by matrix multiplication where the entire matrix is required. This makes such an approach less effective. Considering the varied-length KV cache tensors, PagedAttention [14] dynamically allocates KV cache at the block granularity and can swap partial KV blocks between the CPU and GPU memory. However, the computation has to first wait for the loading/unloading of the KV cache to complete, which compromises the inference performance.

In this paper, we propose a new computing paradigm, named *TwinPilots*, where both GPU and CPU are employed and the inference job is executed in a multi-processor-like system. The idea of introducing a parallel-computing model between CPU and GPU is usually dismissed for deep learning applications as the CPU has a relatively negligible computing power. However, for the inference task on a memory-constrained GPU server, the idea represents a misconception as the GPU speed has been much slowed by the data loading via the PCIe bus. When input data mostly resides in the CPU memory and needs to be constantly retrieved into the GPU memory for the stalled GPU to resume its computation, a new processor-memory architecture can be visualized. As shown in Figure 1, the CPU memory is the main memory, the PCIe bus is the GPU's memory bus, and GPU memory becomes its cache. Situated beside the GPU hierarchy is the traditional CPU-cache-memory structure. In this new structure, both GPU(s) and CPU run in parallel to collaboratively perform their designated inference computation. In the TwinPilots computing paradigm, the original GPU compute-IO scheduling problem is translated into a load balancing problem on a multi-processor execution platform. It paves the road to a solution involving CPU and maximally alleviating the bottleneck at the PCIe bus. It effectively defines the optimization objective (load balancing) so that the inference throughput can be improved without compromising its response time.

In the design, we break down each transformer layer into a set of constituent operations and calculate their respective input data size. We also estimate the operations' CPU execution time and the data's loading time (into the GPU memory). Relatively, the GPU execution time and CPU data loading time are negligible. With this information, Twin

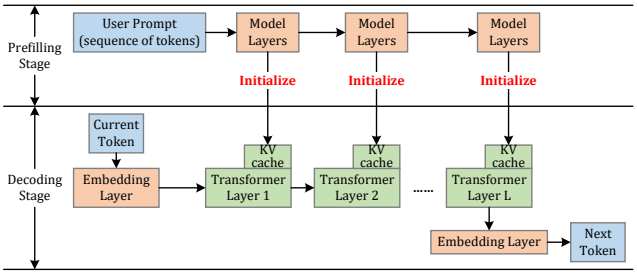


Figure 2: The LLM inference process

Pilots can schedule the operations between the GPU and CPU to achieve a load-balanced execution. As the number of user prompts and prompt length vary, TwinPilots has an online execution scheduler that tracks history execution information and then uses the collected information to determine the processor for an operation to be carried out. We have implemented TwinPilots based on PyTorch [20] and integrated it into several well-known large language models. We extensively evaluate it on a server with an NVIDIA A10G GPU and AMD EPYC 7R32 CPU. Experiment results show that TwinPilots achieves 3.39×, 2.15× and 2.83× inference throughput speedup over FlexGen on Llama-13B, Llama-33B [27] and OPT-30B [32], respectively. Meanwhile, compared with FlexGen, the amount of data transfer is reduced by up to 72.1% and 56.9% on Llama-33B and OPT-30B, respectively. To support increasingly larger models and further show the effectiveness of GPU-CPU co-computation, we allow the TwinPilots platform to include more than one GPU and apply both pipeline parallelism and tensor parallelism in the parallel model execution. The experiments on a server with 4 NVIDIA A10G GPUs show that TwinPilots can achieve up to 2.08× throughput speedup compared with FlexGen on Llama-70B [28].

2 BACKGROUND AND MOTIVATION

2.1 Large Language Model

The inference process of a large language model involves utilizing the trained model to generate outputs based on a given input text (i.e., prompt). This generation process comprises two stages: *prefilling* and *decoding*, as depicted in Figure 2. In the prefilling stage, the user prompt (i.e., a sequence of tokens) is fed into the model and initializes the KV cache for each transformer layer. The KV cache is essential for the subsequent decoding stage. During the decoding stage, the model uses the KV cache to generate output tokens sequentially. After a token is generated, it is fed back into the model and updates the KV cache. The updated KV cache is then used to generate the next token. This process repeats itself until the model generates a special token `<End-Of-Sentence>`, signifying completion of token generation, or the limit on the output sequence length is reached.

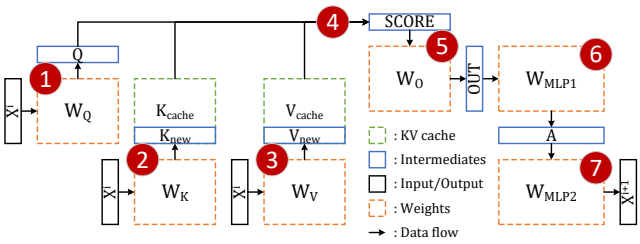


Figure 3: Transformer Layer.

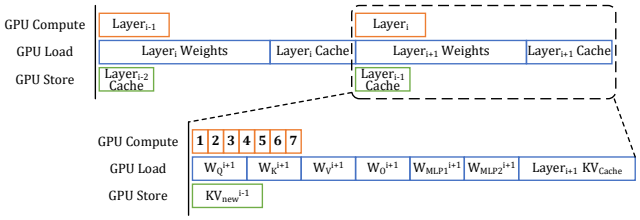


Figure 4: Existing execution pipeline.

Figure 2 includes some details of the decoding stage. With each layer's output serving as the input of the next layer, a typical LLM model starts with an input embedding layer, followed by multiple transformer layers, and concludes with an output embedding layer. Specifically, an embedding layer performs row indexing to its weight matrix to explore input features [34]. With the embedding layer, the following transformer layers can understand the relationship between inputs. A transformer layer performs several matrix multiplications, dominating the latency of the time-consuming decoding stage [29]. Therefore, in this paper, we mainly focus on optimizing transformer layers in the decoding stage for higher inference performance.

2.2 The Transformer Layer

The basic data flow in a transformer layer is shown in Figure 3. Each transformer layer is divided into seven operations according to their generated results: Q, K, V, SCORE, OUT, MLP1, and MLP2 described in Table 1.

Table 1: Operations in transformer layer.

Operation	Description	Dependency
Q	$q = X^i \times W_Q^i$	-
K	$k = X^i \times W_K^i$	-
V	$v = X^i \times W_V^i$	-
SCORE	$score = \text{Attention}(q, \text{APPEND}(K, k), \text{APPEND}(V, v))$	**Q,K,V**
OUT	$O = score \times W_O^i$	**SCORE**
MLP1	$A = O \times W_{\text{MLP1}}^i$	**OUT**
MLP2	$X^{i+1} = A \times W_{\text{MLP2}}^i$	**MLP1**

A common practice to extend limited GPU memory is to place the weights and KV cache that cannot be accommodated in the GPU memory to the host CPU's memory and load them to the GPU memory on demand when performing

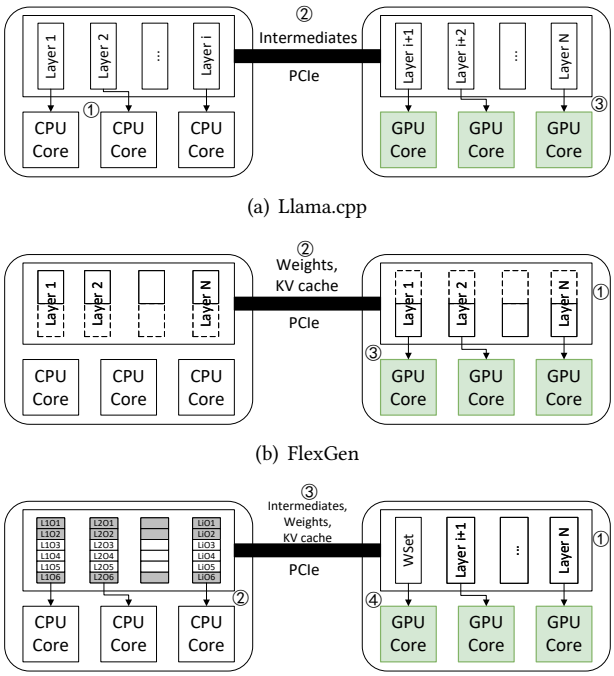


Figure 5: Comparison for different offloading methods. 5(a) Layer 1 to Layer i are placed in the CPU memory and computed on the CPU cores (①), and Layer $i+1$ to Layer N reside in the GPU memory and are computed on the GPU cores (③). Only the intermediate data generated by Layer i (②) is transferred between the CPU and GPU. 5(b) A subset of Layer 1 to Layer N's weights is pinned into the GPU memory (①), and the rest are placed in the CPU memory. During the inference, each layer is loaded into the GPU memory (②) and computed on the GPU cores (③). 5(c) TwinPilots pins several layers into GPU memory and reserves a small working set (①). It splits the remaining layers into several operations. During the inference, gray-colored operations' weights are loaded into the GPU memory (③) and computed on the GPU cores (④). In the meantime, white-colored operations are computed locally on the CPU (②). And only the generated results are transferred to the GPU (③).

inference. However, the limited PCIe bandwidth causes the data stall problem. Most offloading frameworks employ the inter-layer pipeline, aiming to overlap the GPU computation of $Layer_i$, the data loading of $Layer_{i+1}$, and the KV cache store of $Layer_{i-1}$ in a pipelined way, as illustrated in Figure 4. However, the pipeline is bottlenecked by loading large weights and KV cache, resulting in GPU under-utilization and therefore limiting the inference throughput. Even worse, since different transformer layer has different weights and KV cache, the data for each layer to take a considerable amount of time to load is only active for a short time period. It then quickly becomes obsolete and is released. It is of vital importance to reduce the amount of loaded data to mitigate the pipeline bottleneck.

2.3 Existing Offloading Methods

There are a number of schemes that offload data to the CPU memory to alleviate the GPU memory constraint. Among them, vLLM is a popular LLM serving framework that was designed based on PagedAttention [14], which dynamically allocates and manages the KV cache in the block granularity. Partial KV cache can be temporarily swapped out of GPU memory to make room for new requests. However, it requires that model weights fit into the GPU memory, which is not feasible on machines with limited resources. Moreover, it serializes KV cache swapping and model inference. Thus it cannot exploit the opportunity of computation overlapping, compromising the inference throughput.

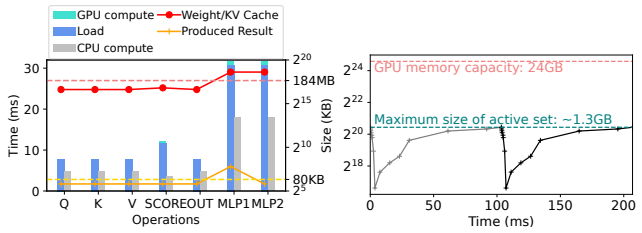
Llama.cpp [7] is a lightweight LLM inference framework designed for various hardware to support CPU inference. However, Llama.cpp adopts static layer partitioning for oversized LLMs. As shown in Figure 5(a), it places some contiguous layers' weights in the CPU memory and the remainings in the GPU memory. During an inference, it first uses the CPU to compute the CPU-resident layers and then uses the GPU to compute the GPU-resident layers. This approach minimizes data communication between CPU and GPU. However, it underutilizes both CPU and GPU.

FlexGen is a GPU-centric inter-layer pipeline method aiming to achieve higher generation throughput. As shown in Figure 5(b), for each layer, it pins a portion of weights in the GPU memory. During the inference, it hides data movement latency by overlapping the GPU computation of the current layer and the data loading of the next layer. As the data loading time usually far exceeds the GPU computation time, this arrangement may make the GPU seriously underutilized.

Existing methods either disregard the potential of utilizing the CPU to assist the LLM inference or cannot achieve a balanced CPU computation load and PCIe transfer load, resulting in either GPU or CPU under-utilization.

2.4 The Motivation

Since most data resides in the host memory, utilizing the host CPU for certain operations in a transformer layer can be a potential way to relieve the load on the PCIe bus. Thus, in this subsection, we first dissect the time cost of each operation, followed by a comparison of the sizes between the transferred data (required by each transformer operation) and the generated results to show the performance improvement potential of involving CPU in the LLM inference. Subsequently,



(a) Time breakdown and data size (b) Active tensor size

Figure 6: Transformer layer analysis of OPT-30B (the prompt length is 896 and decode length is 128).

we monitor the GPU memory occupied by active tensors during the inference, leaving a larger design space for us to explore more efficient memory usage.

Time analysis of transformer layer. We have conducted the experiment with the same setup as that in Section 5 and a batch size of 4 on the OPT-30B. The results are shown in Figure 6(a), which indicate a significant potential of leveraging CPU to accelerate the LLM inference. It is noteworthy that not only the data loading time does significantly surpass the GPU computing time, but also there is a substantial reduction in data size after completing each operation. Since the input and output tensors of each operation are much smaller than the weights/KV cache involved in the computation, if some operations can be reversely "offloaded" to the host CPU, we can transfer only a small amount of intermediate results instead of large weights/KV cache to the GPU, significantly reducing the amount of loaded data.

Working set analysis of LLM inference. The working set in the GPU memory can be regarded as the active tensors that are used by the current layer's computation plus the pre-allocated space for the next layer's weights and KV cache. As illustrated in Figure 6(b), the active tensor size is far less than the GPU memory capacity during the inference. The sudden drop of the active tensor size shown in the figure indicates the completion of the current layer's computation, after which its weights and KV cache can be freed. It leaves us a certain amount of available memory space besides the pre-allocated space for the next layer, which needs to be judiciously used to improve inference performance.

New challenges. Several challenges need to be addressed to fully exploit opportunities from both the underutilized CPU and the remaining GPU memory beyond the working set. The primary concern is to determine which operations should be executed on the CPU, as each operation incurs extra data loading time when executed on the GPU while consuming more computing time when executed on the CPU. We model this as a load balancing problem in Section 3.1, and accordingly propose our online scheduler in Section 3.2. We also introduce a new data placement strategy in Section 3.3, taking full use of the remaining GPU memory to relieve PCIe pressure. We then give our runtime execution pipeline in Section 3.4 and discuss some optimizations in Section 3.5.

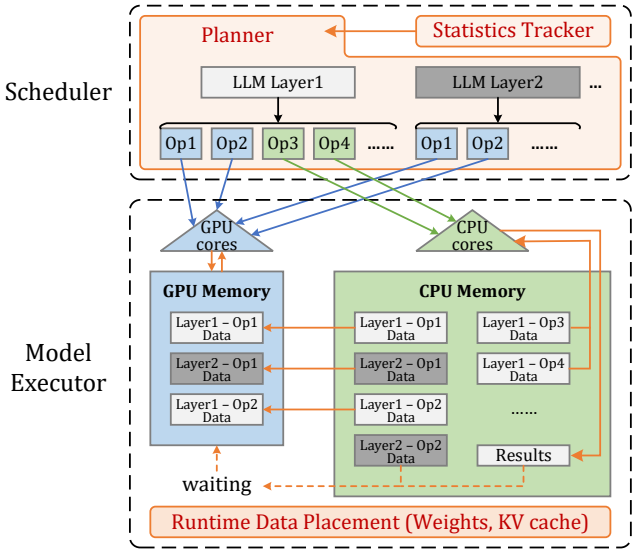


Figure 7: TwinPilots overview.

3 THE DESIGN OF TWINPILOTS

In this section, we present TwinPilots, a novel computing paradigm that leverages both GPUs and CPUs for LLM inference. As shown in Figure 7, TwinPilots comprises an online scheduler and a model executor. The online scheduler collects historical execution information from the statistics tracker and generates execution plans (e.g., computing processors) for each operation. The model executor then performs data prefetching and operation computation based on the plan, utilizing both CPU and GPU resources in a pipelined manner.

3.1 Load Balancing with CPU Computation

Our fine-grained segmentation of transformer operations enables accurate measurement of the CPU execution time (when executing on the CPU) and the GPU data loading time (when executing on the GPU). Specifically, we analyze operations such as Q, K, V, SCORE, OUT, MLP1, and MLP2. It is worth noting that due to the CPU's limited computational capability, scheduling an operation on the CPU results in a longer execution time. While this approach conserves PCIe bandwidth by avoiding large weight and KV cache transfers to the GPU, it necessitates careful consideration of CPU computing time and GPU data transfer time balancing.

Figure 8(a) illustrates a common computing pipeline, with all needed data being loaded to the GPU for each operation's execution. When we relocate three operations (Q, K, V) to the CPU (as shown in Figure 8(b)), the increased CPU computation time is still shorter than the time required to load W_O, W_{MLP}, and *Cache* to the GPU. However, when we move

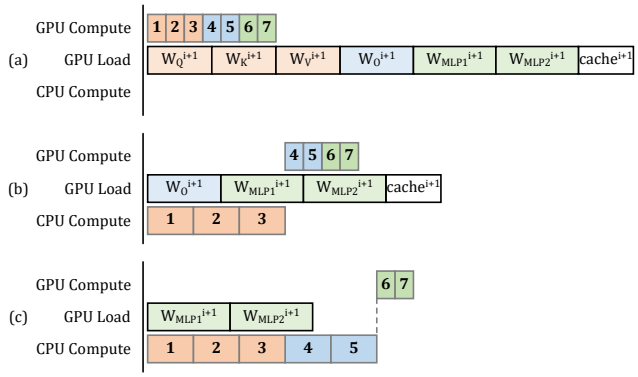


Figure 8: Offloading pipelines w/o store streams.

two additional operations SCORE and OUT (Figure 8(c)), the CPU computation time exceeds the GPU data loading time (considering that the loading time of the intermediate results generated by CPU computation is negligible compared to the reduced weights and KV cache movement).

Since the final execution time is dominated by the longer one between the total data loading time and total CPU computing time, we must select the computation device for each operation based on the shortest makespan policy. This can be modeled as a load-balancing problem on unrelated machines [15], which has been proven to be NP-Complete [22].

3.2 The Online Scheduler

Since the LLM will serve different numbers of user prompts, and the sequence length varies during the token generation, the execution plan must change for different batches of requests. To address this, we propose an online scheduler in this subsection, which is designed to generate an offloading plan for all operations in the transformer layers.

The online scheduler comprises a statistic tracker and a planner. The tracker gathers historical timing information from the model executor, including the execution time for each operation on both CPU and GPU (including data loading time) across various batch sizes and sequence lengths. When the scheduler attempts to schedule a request, it calculates the average estimated execution time for each operation with the following time complexity: 1) for the SCORE operation, the KV cache loading time has a complexity of $O(bs)$ and the CPU computation time also follows $O(bs)$; 2) for other operations, the weight loading time has a complexity of $O(1)$ and the CPU computation time is $O(b)$.

Problem Description. Given a set **ops** containing N operations and 2 devices: GPU and CPU, the processing time of operation i is gpu_i on GPU and cpu_i on CPU. Let $device_i$ denote the computation device of operation i (1 for CPU and 0 for GPU), $S_{cpu} = \sum_{i=1}^{N} cpu_i device_i$ and $S_{gpu} = \sum_{i=1}^{N} gpu_i(1 - device_i)$ be the total CPU processing time and GPU processing time of a certain plan $D = \{device_i; i = 1, \ldots, N\}$, respectively. The makespan λ of the plan is $\max(S_{cpu}, S_{gpu})$. The objective is to find a plan D that minimizes the λ. We give the linear programming $LP(\lambda)$ as:

$$\begin{aligned} &\text{Minimize} && \lambda \\ &\text{Subject to} && \textstyle\sum_{i=1}^{N} cpu_i device_i \le \lambda \\ & && \textstyle\sum_{i=1}^{N} gpu_i(1 - device_i) \le \lambda \\ & && device_i \in \{0, 1\} \qquad \forall\, i \in 1, \ldots, N \end{aligned}$$

By utilizing linear programming, our planner can find the balanced execution plan with the shortest makespan, thereby reducing the pipeline length.

Algorithm 1 Online Planner

Require: Batch size b, Sequence length s, History records R_{cpu} = {Q: {"b": 4, "s": 134, "time": ...}, ...}, R_{gpu} ={...}

Require: Operations **ops** = {Q, K, V, SCORE, OUT, MLP1, MLP2}

1: /* *Estimate the loading time and CPU computation time for each operation* */
2: **for** op ∈ ops **do**
3: **if** op is SCORE **then**
4: $gpu_i = b \times \text{Avg}\left(\frac{R_{gpu}[\text{SCORE}].\text{time}}{R_{gpu}[\text{SCORE}].\text{b} \times R_{gpu}[\text{SCORE}].\text{s}}\right)$
5: $cpu_i = b \times \text{Avg}\left(\frac{R_{cpu}[\text{SCORE}].\text{time}}{R_{cpu}[\text{SCORE}].\text{b} \times R_{cpu}[\text{SCORE}].\text{s}}\right)$
6: **else**
7: $gpu_i = \text{Avg}\left(R_{gpu}[\text{op}].\text{time}\right)$
8: $cpu_i = b \times \text{Avg}\left(\frac{R_{cpu}[\text{op}].\text{time}}{R_{cpu}[\text{op}].\text{b}}\right)$
9: /* *Solve linear programming* */
10: SOLVE($LP(\lambda)$)
11: **for** $i = 1, \ldots, N$ **do**
12: **if** $device_i == 0$ **then**
13: op.*device* = GPU
14: **else**
15: op.*device* = CPU

As shown in Algorithm 1, given the request information and history statistics, the planner first estimates the time (lines 2-8), and then uses the estimation time to solve the linear program to generate the plan (lines 10-15).

Statistics records are initiated by running several predefined batch sizes and sequence lengths offline. The statistics tracker periodically aggregates historical information to reduce the overhead of calculating the estimation time. Although the load balancing problem is NP-complete, the small solution space (the few number of operations) ensures that the overhead introduced by the online scheduler remains negligible even when aggragating the records at the end of each decoding step.

3.3 Data Placement

LLM weights and KV cache exhibit distinct memory read/write attributes. The weight matrices are read-only,

whereas the KV cache is writable and append-only. Consequently, we employ separate placement strategies for them.

3.3.1 Weight Placement. As discussed in Section 2.1, transformer-based LLM consists of two embedding layers and many transformer layers. Based on their different computation and memory access patterns, we have different placement and computation strategies.

Embedding layers. The embedding layer performs row indexing to its weight matrix, which has a sparse memory access pattern. There is no need to copy the weight to the GPU memory for each iteration. Therefore, the weight of the embedding layer resides in host memory and we adopt Direct-Host-Access [12] to access it on demand at page granularity.

Transformer layers. As discussed in Section 2.4, the LLM inference pipeline only requires a small memory working set, leading to a substantial unused GPU memory space. Therefore, we categorize transformer layers into pinned and unpinned ones, utilizing the remaining GPU memory and the memory working set, respectively. For pinned layers, all weights reside in GPU memory to achieve maximum inference speed. In TwinPilots, we pin the first few transformer layers of LLM and always assign GPU for their computations. For weights of unpinned layers residing in host memory, our offloading strategy (Section 3.2) is adopted to balance their CPU computation time and GPU data loading time.

Algorithm 2 Offloading pipeline

```
   /* For each transformer layer */
2: for i = 1, ..., l do
       /* Load next layer's weights/KV cache */
4:     for op ∈ Layer_{i+1}.ops do
           if op.device == GPU then
6:             if op == SCORE then
                   /* Evict old KV cache blocks, swap in
                      required blocks. */
8:                 FIFO_SWAP(op.KVCache)
               else
10:                LOAD_WEIGHT(op.weight)
       /* DAG Execution */
12:    for op ∈ Layer_i.ops do
           /* Wait for dependent operations to complete. */
14:        op.device ← WAIT_FOR(op.dependency)
           /* Compute operations for the current layer. */
16:        op.device ← COMPUTE(op)
       /* Wait for GPU computation tasks */
18:    SYNC()
```

3.3.2 Dynamic KV Cache Placement. To meet the requirement for varied sequence lengths, TwinPilots dynamically allocates and manages the KV cache at block granularity, which is similar to vLLM. We reserve a First-In-First-Out (FIFO) GPU buffer to store partial KV cache blocks, thus shortening the KV cache loading time and speeding up the inference. The sequence length provided to the scheduler is `BlockSize`×`#CPUBlocks`, where `#CPUBlocks` is the number of blocks residing in CPU memory.

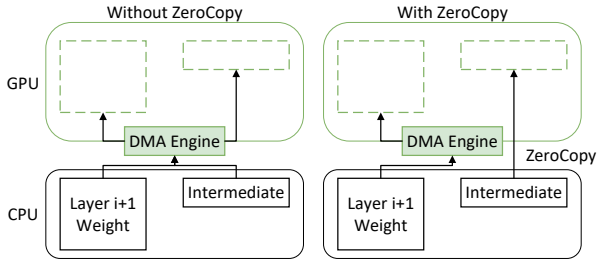


Figure 9: Zero copy optimization.

3.4 Offloading Pipeline

The computation flow of each transformer layer can be abstracted as a Directed Acyclic Graph (DAG), where each operation has multiple dependencies as depicted in Table 1. Each operation depends on the completion of its prerequisite operations. With an abstracted computation flow based on the generated offloading plan, TwinPilots automatically issues the computation of each operation to corresponding devices during inference while maintaining the dependency, which is described in Algorithm 2. By overlapping 1) the loading of the next layer's weights, 2) KV cache swapping between the next layer and the previous layer (lines 4-10), and 3) the current layer's computation on both CPU and GPU (lines 12-16), TwinPilots can achieve near-optimal efficiency of the offloading pipeline.

3.5 Optimization for Intermediate Transfer

The GPU DMA engine is usually non-preemptive: for each direction, later-issued DMA transfers are blocked until all early-issued transfer tasks are completed. Unfortunately, this characteristic introduces an unpredictable delay in our intermediate result transfer, which may be blocked by background weight matrices pre-fetching.

To address this issue, we utilize the **Zero-Copy** mechanism in NVIDIA GPUs, which allows direct PCIe read/write from GPU cores to page-locked CPU memory. In this case, the intermediate result transfer would not be blocked by the background DMA transfer of model weights. As a result, it reduces the delay of intermediate result transfer and enhances scheduling performance.

4 IMPLEMENTATION

4.1 Single GPU

We implement TwinPilots based on FlexGen, a popular pipeline-based GPU-centric LLM inference framework designed for a single GPU.

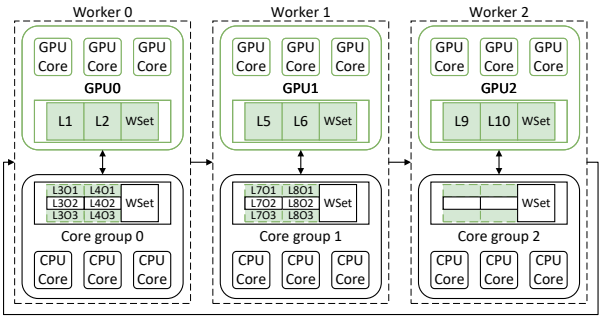


Figure 10: TwinPilots for Pipeline Model Parallelism

Zero copy. The zero copy of intermediate tensors accessed from GPU is realized by utilizing NVIDIA Unified Addressing. We create a PyTorch extension where we can register the underlying storage of CPU tensors using `cudaHostRegister`, enabling direct access from GPU cores to the host memory.

GPU FIFO buffer for KV cache. Each transformer layer has a GPU FIFO buffer to store loaded KV cache blocks. To avoid cache consistency issues, the FIFO buffer employs a write-through policy, in which appending a new KV cache to it will also trigger background write to the copy of the CPU KV cache. The synchronization happens at the end of each decoding step. Considering the abundant PCIe bandwidth for GPU-issued writes to the CPU, the overhead is negligible. Therefore, the CPU has a complete KV cache while the GPU maintains a partial copy. When the planner tries to estimate the CPU execution time and GPU execution time of SCORE operation, it does not need to consider moving the latest GPU KV cache back to the CPU.

4.2 Multiple GPUs

To efficiently handle multiple batches of user requests and enable ever-increasing language models, TwinPilots is also implemented to support multiple GPUs. There are two major distributed model parallel methods: pipeline parallelism and tensor parallelism [25], which partition the model along layers and weights, respectively. The idea of TwinPilots's GPU-CPU co-computation can be adapted to both.

4.2.1 Pipeline Model Parallelism. To support pipeline model parallelism on multiple GPUs, TwinPilots partitions CPU cores into distinct core groups. As shown in Figure 10, each GPU collaborates with one CPU core group to form a worker, which handles a portion of consecutive layers. LLM layers are evenly distributed across workers. Each worker has its own working set and works asynchronously with other workers, running an instance of model executor on its assigned layers, and passing intermediate results to the subsequent worker.

4.2.2 Tensor Model Parallelism. We also implement TwinPilots with tensor model parallelism, which partitions the weight matrices along the row or column dimension at a finer granularity. Figure 11 illustrates an example of a column-based partition. Workers are categorized into GPU Workers and CPU workers based on their assigned computation resources. The GPU-Worker's memory consists of pinned weights, KV cache blocks, and a working set. The memory of CPU-Worker stores assigned partitions of weights and KV cache blocks. During inference, the I/O engine of GPU-Workers prefetches each operation's unpinned weights and swaps KV cache blocks. The CPU-Worker performs computation concurrently to hide the prefetching latency. The weight partition is determined by the speed variance between PCIe bandwidth and CPU computing capability.

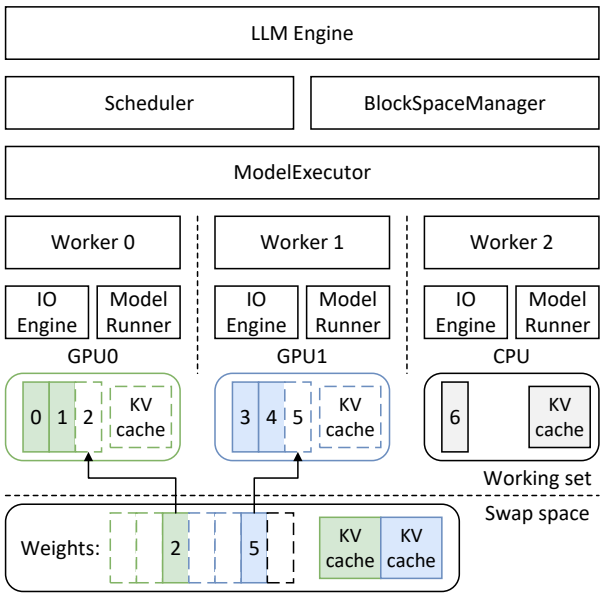


Figure 11: TwinPilots for Tensor Model Parallelism

Two major collective communication operations in tensor model parallelism are commonly used to share the intermediate results: all-gather and all-reduce. We have implemented those two operations using the NVIDIA zero copy mechanism to enable parallel intermediate collection and weight matrices prefetching.

5 EVALUATION

5.1 Experiment Setup

Table 2: Hardware Settings.

CPU	AMD EPYC 7R32 2.8GHz, 24 cores [1]
SIMD	AVX256 [10]
Memory	256GB DDR4
GPU	4× NVIDIA A10G 24G GDDR6
Interconnect	PCIe 3.0×16

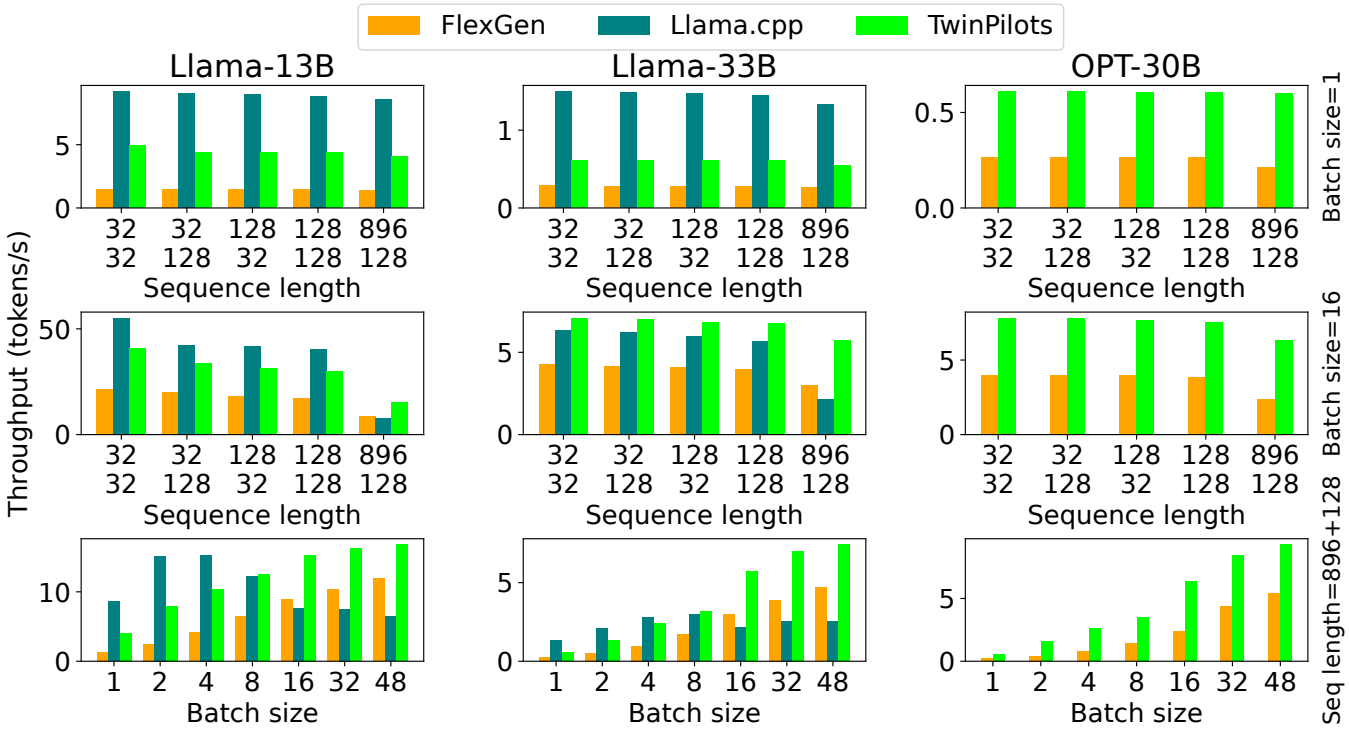


Figure 12: Decoding throughput on various LLMs with different sequence lengths and batch size settings.

The server hardware settings are shown in Figure 2. Our server runs Linux 6.5 with CUDA 12.1 [2]. We implement our method on PyTorch 2.3 [20] with Intel-MKL [11] and OpenMP [19] support to speed up CPU computation. We evaluate our method on widely used open-source LLMs including OPT-30B, OPT-66B, and Llama-30B represented in FP16 format with different batch sizes and sequence lengths.

We first conduct experiments using a single GPU, comparing TwinPilots with FlexGen, our baseline method, showing our performance gains in decoding throughput and data transfer. Subsequently, we perform a detailed breakdown analysis of several TwinPilots components. Next, we evaluate the throughput and user response latency of FlexGen and TwinPilots on multiple GPUs. Finally, we compare TwinPilots to the state-of-the-art LLM framework Llama.cpp, which features CPU inference.

5.2 Decoding Throughput

Sequenth length. We first evaluate the decoding throughput of TwinPilots across different sequence length settings on various LLMs. To exhibit the performance of TwinPilots in different language tasks such as chatbot, QA, and text summarization, we first compare TwinPilots with FlexGen using the single prompt with various prompt lengths ranging from 32 to 896, while the decoding lengths range from 32 to 128. For FlexGen, all computations are accomplished on the GPU, which employs pipelined inference across three consecutive transformer layers. For TwinPilots, the KV cache dynamic allocation granularity is 32 tokens. To better show the effectiveness of coordinate GPU-CPU computation, the size of the GPU KV cache FIFO buffer is set to zero, meaning that the KV cache resides in the CPU memory. Both schemes reserve only the working set memory, and use the remaining GPU memory to pin model weights.

The decoding throughput is computed as $\frac{\text{decode length}}{\text{decoding time}}$. As shown in Figure 12, TwinPilots outperforms FlexGen under all sequence length settings on different models. It achieves up to 3.39×, 2.15×, and 2.83× throughput speedup on Llama-13B, LLama-33B, and OPT30B, respectively. This is attributed to our online planner, which can assign more operations to the CPU to save data loading time and shorten the pipeline, thus improving inference throughput.

Batch size. TwinPilots also exhibits impressive performance across different batch sizes. As depicted in Figure 12, we compare TwinPilots with FlexGen at different batch sizes ranging from 1 to 48. Prompts within the same batch are not shared. TwinPilots consistently outperforms FlexGen under all batch sizes, which demonstrates TwinPilots's effectiveness at different batch sizes. It is worth mentioning that, as the batch size increases, for each operation (excluding SCORE) the CPU execution time grows linearly since the computation demands scale linearly with the batch size. In contrast, the GPU execution time remains nearly unchanged, primarily because it is still dominated by the weight loading

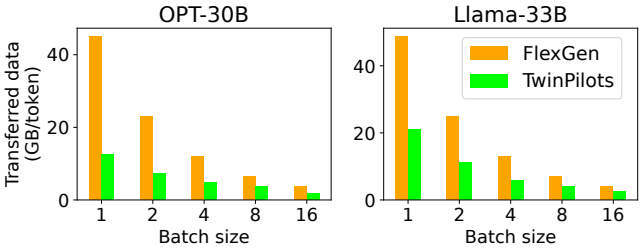


Figure 13: Amount of transferred data via PCIe per generated token (prompt length=896, decode length=128).

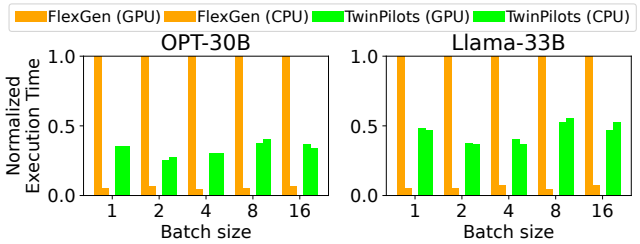


Figure 14: GPU-CPU load balancing (prompt length=896, decode length=128).

time, which is constant. This will reduce the performance improvement over FlexGen, especially when more operations are scheduled onto the GPU. In the worst case, it may degrade to have the same execution pattern as FlexGen, where each operation's CPU time exceeds the total data loading time of all operations. However, such scenarios occur only for ultra-large batch sizes and long sequences, which is beyond the serving capability of a single GPU server. Even for a batch size of 48 with 1K sequence length, TwinPilots still outperforms FlexGen by the speedup of $1.58\times$ and $1.71\times$ on Llama-33B and OPT-30B, respectively.

Moreover, we compare TwinPilots with Llama.cpp, which also features CPU computation. Detailed comparison is discussed in Section 5.9, as their execution strategies are totally different, resulting in diverse favorable scenarios.

5.3 Data Transfer

We measure the amount of data transferred through the PCIe channel by Intel-PCM [9] on OPT-30B and Llama-33B. As illustrated in Figure 13, TwinPilots significantly reduces the amount of transferred data by up to $\approx 72.1\%$ and $\approx 56.9\%$ per generated token compared with FlexGen on OPT-30B and Llama-33B, respectively. This reduction is attributed to its pinning strategy that increases the amount of pinned weights and its offloading strategy that reduces the amount of transferred weights and KV cache.

5.4 CPU-GPU Load Balance

All existing offloading pipelines overlap GPU time (data loading time + GPU computing time) and CPU computing

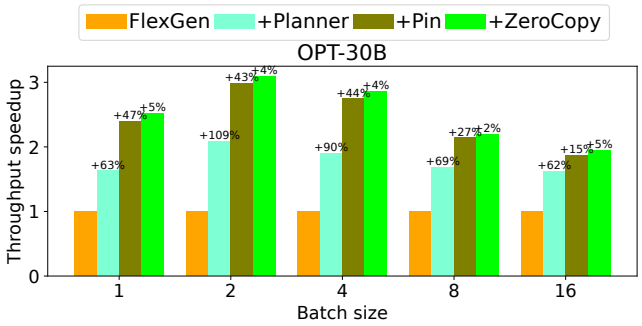


Figure 15: Performance breakdown.

time, aiming to shorten the longer one. We take FlexGen as the baseline to show the effectiveness of TwinPilots's load-balancing effectiveness. Figure 14 shows the normalized pipeline execution time on GPU and CPU, where the total execution time is determined by the longer one. TwinPilots achieves a more balanced load distribution between GPU and CPU compared to FlexGen on both OPT-30B and Llama-33B, reducing the overall execution time.

5.5 Performance Breakdown

We conduct a breakdown analysis of TwinPilots on OPT-30B with prompt length of 128 and decode length of 128. The baseline `FlexGen` employs its own pinning strategy and performs all the computations on the GPU. `+Planner` means adding our proposed load balancing planner to generate an execution plan for each operation of the transformer layer, without pinning any model weights. Additionally, `+Pin` refers to employing the data placement strategy described in Section 3.3. Finally, `+ZeroCopy` means involving the zero copy optimization for intermediate transfer discussed in Section 3.5, instead of using the DMA copy.

As depicted in Figure 15, TwinPilots's load balancing planner exhibits significant performance improvement compared to FlexGen. Even without pinning any layers, TwinPilots can beat FlexGen across various batch sizes, highlighting the substantial potential of leveraging the CPU to reduce data loading and shorten the pipeline. Additionally, the improvement achieved through zero-copy optimization indicates reduced latency in transferring small intermediate results.

5.6 GPU KV Cache Buffer

We evaluate the performance impact of varying GPU KV cache buffer size on Llama-33B. Specifically, we explore different buffer size proportions relative to the maximum KV cache size, ranging from 0% to 100%.

As shown in Figure 16, when the batch size is small, the throughput is insensitive to the buffer size changes. However, for large batch sizes, increasing the proportion may squeeze the GPU memory space of pinned layer weights, which may result in a performance drop. For instance, at the batch size

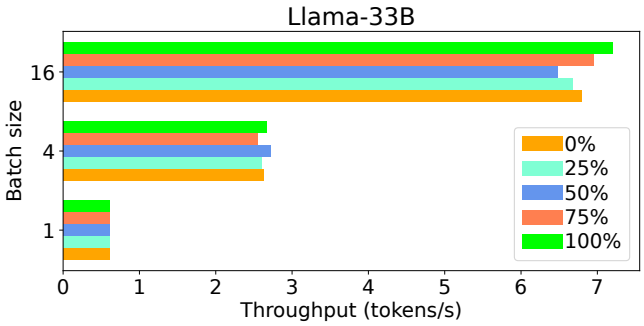


Figure 16: Different GPU KV cache buffer size (prompt length=128, decode length=128) .

of 16, when the buffer proportion changes from 0% to 50%, the SCORE operation is scheduled to execute on the CPU, but the number of pinned layers decreases, hence hurting the throughput. On the other hand, increasing the buffer size can also shorten the GPU execution time of the SCORE operation, which may potentially improve the performance. For instance, at the batch size of 16, when the buffer proportion changes from 75% to 100%, the SCORE operation is scheduled onto GPU, resulting in increased throughput due to faster SCORE execution.

5.7 Overhead Breakdown

We analyze the overhead of different components. In Figure 17, `Planner` means the overhead incurred by estimating each operation's execution time on different processors and the linear problem solver to generate the execution plan. `Intermediate` means the overhead caused by transferring intermediate results between different operations. `Idle` means the average pipeline idle time. For example, if the CPU finishes its task quickly, it may wait until the next layer's data is ready, and vice versa. `Exec` means the useful computation time (including data loading time for GPU operations).

As shown in Figure 17, the planner incurs negligible overhead ($< 1\%$) across different batch sizes and models, even though it is dynamic. Regarding intermediate transfer, as the batch size increases, the size of intermediate results also grows, leading to more PCIe read/write issuing overhead, which incurs a larger proportion of intermediate transfer (from $< 1\%$ to $\approx 10\%$). However, this increase remains insignificant compared to the total execution time. Additionally, the average idle time is small ($< 5\%$) across different batch sizes, demonstrating the effectiveness of our load-balancing planner.

5.8 Multiple GPUs

TwinPilots can be adopted with multiple GPUs to support increasingly larger language models. We evaluate TwinPilots

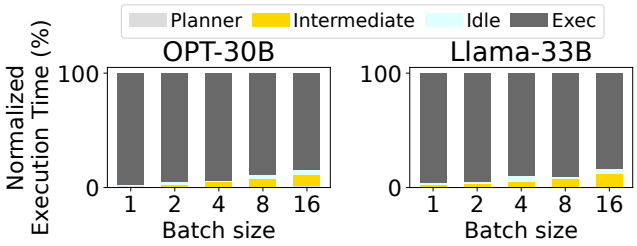


Figure 17: Overhead breakdown.

performance for multiple GPUs on Llama-70B with both prompt length and decode length equal to 128. For `FlexGen`, we modified it to support multiple GPUs: each GPU handles several consecutive layers, employing its original pinning and computing strategy. Multiple GPUs together adopt pipeline model parallelism. For `TwinPilots-PP`, it follows the strategy described in Section 4.2.1. For `TwinPilots-TP`, it employs tensor model parallelism described in Section 4.2.2, which is based on vLLM. Notably, Llama-70B requires about 140GB of memory represented in FP16 format to hold its weight matrices. Consequently, the original vLLM cannot run Llama-70B even on 4× NVIDIA A10G 24GB, since it does not support weight offloading.

Figure 18 shows that TwinPilots-PP exhibits significantly higher decoding throughput compared with FlexGen and TwinPilots-TP. TwinPilots-PP also demonstrates superior scalability with respect to the number of GPUs. This is attributed to both the load-balancing planner design and the resource partition strategy of GPUs and CPU core groups.

Table 3: Comparison of user response latency for multiple GPUs (in seconds). Batch size=4.

#GPUs	FlexGen	TwinPilots-PP	TwinPilots-TP
1	11.799	**5.435**	5.952
2	8.154	5.287	**3.454**
3	7.290	4.386	**2.429**
4	7.185	3.566	**2.033**

Although TwinPilots-TP shows lower throughput than TwinPilots-PP, it significantly reduces user response latency compared to both TwinPilots-PP and FlexGen, as shown in Table 3. The user response latency is defined as the time interval between the generation of the current token and the next token for each user session, which better reflects the user experience. Benefitting from the shortened pipeline, TwinPilots-PP reduces the response latency by up to 53.9% compared with FlexGen. Featured by more fine-grained tensor model parallelism, TwinPilots-TP further outperforms TwinPilots-PP in terms of user response latency. While pipeline model parallelism may encounter reduced throughput due to bubbles (potentially caused by user session suspension during real-time interactions), it performs optimally when handling aligned batched LLM inferences. In

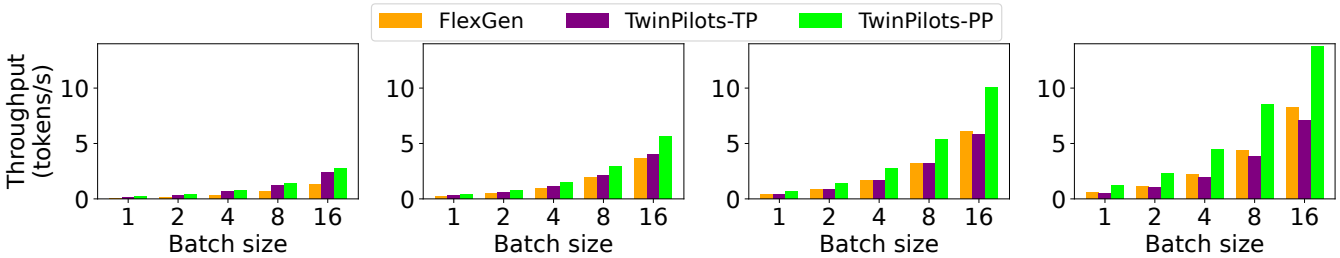


Figure 18: TwinPilots for Multi-GPU on Llama-70B.

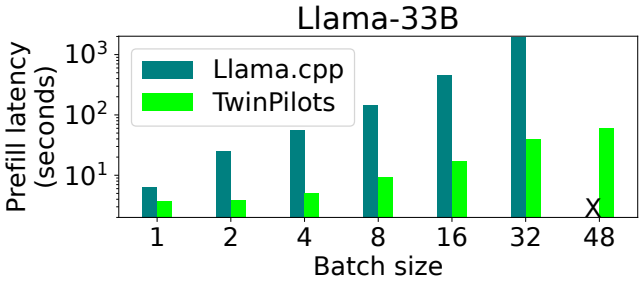


Figure 19: First token latency comparison between Llama.cpp and TwinPilots.

contrast, tensor model parallelism can provide more stable throughput across diverse use cases.

5.9 Comparison with Llama.cpp

As shown in Figure 12, we compare TwinPilots with Llama.cpp across different batch sizes and sequence lengths. Prompts within the same batch are not shared. While Llama.cpp demonstrates impressive performance for small batch sizes, it falls short compared to TwinPilots when the batch becomes larger. The throughput of Llama.cpp decreases quickly as the sequence length increases, finally surpassed by TwinPilots. Furthermore, the performance degradation of Llama.cpp is more severe on larger models.

As discussed above, Llama.cpp performs statical partitioning across model layers, where the only communication between the CPU and the GPU takes place when the last CPU-resident layer passes hidden states to the first GPU-resident layer. This design choice contributes to its significantly higher inference throughput for smaller batch sizes and shorter sequence lengths. Moreover, when the computational load is not heavy, the efficiency of different tensor operation implementations, language interpreter, and runtime overhead are critical, which is the major reason why TwinPilots underperforms Llama.cpp for small batch sizes. However, when the batch size and sequence length increase, the computation on the layers statically allocated to the CPU becomes too much, making the CPU become a performance bottleneck. In such a scenario, the scheduling strategy becomes more crucial. Moreover, Llama.cpp reserves KV cache working space for the maximum sequence length, which takes away the available GPU memory space for pinning layers. In contrast, TwinPilots employs a dynamic load-balancing strategy that scales decoding throughput effectively with large batch sizes.

Furthermore, the heavy CPU computation of Llama.cpp under large batch sizes greatly increases its prefilling latency (i.e. first token latency), a critical factor for user experience. As shown in Figure 19, the prefill latency of Llama.cpp requires several minutes when the batch size exceeds 4 (even reaching hours of latency when the batch size increases to 48), which is an unacceptable delay in practice. TwinPilots consistently achieves a much lower prefill latency than Llama.cpp across various batch sizes.

5.10 Discussion

TwinPilots is advantageous in small to medium-scale LLM deployments, such as proprietary models used in an enterprise that concurrently serve a limited number of users. For example, a company might employ a fine-tuned model based on confidential documents to provide automatic data analysis or problem-solving service [30] for its staff or tailored audience. Under such scenarios, TwinPilots offers scalable decoding throughput while maintaining low first-token latency.

6 CONCLUSION

In this paper, we propose TwinPilots, a new computing paradigm for GPU-CPU co-optimized LLM inference. When dealing with LLMs that exceed GPU memory capacity, TwinPilots reduces the amount of data transfer between the host memory and GPU memory by exploring the host CPU's computation capabilities. By balancing the workload between GPU and CPU, TwinPilots achieves an impressive up to 3.39× inference throughput speedup compared to FlexGen on a single GPU. To accommodate ultra-large LLMs, we extend TwinPilots to multiple GPUs for both pipeline model parallelism and tensor model parallelism, exhibiting a substantial 2.08× throughput speedup on Llama-70B using 4 GPUs.

ACKNOWLEDGMENTS

The work described in this paper is supported by grants from the Research Grants Council of the Hong Kong Special Administrative Region, China (GRF 14219422, GRF 14202123).

REFERENCES

[1] AMD. 2024. AMD EPYC 7002 Series Processors. https://www.amd.com/en/products/processors/server/epyc/7002-series.html

[2] Rob Armstrong. 2022. CUDA Toolkit 12.0 Released for General Availability. https://developer.nvidia.com/blog/cuda-toolkit-12-0-released-for-general-availability/

[3] Zhihao Bai, Zhen Zhang, Yibo Zhu, and Xin Jin. 2020. PipeSwitch: Fast Pipelined Context Switching for Deep Learning Applications. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. USENIX Association, 499–514. https://www.usenix.org/conference/osdi20/presentation/bai

[4] BentoML. 2024. OpenLLM. https://github.com/bentoml/OpenLLM

[5] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, et al. 2023. Palm: Scaling language modeling with pathways. *Journal of Machine Learning Research* 24, 240 (2023), 1–113.

[6] Machine Learning Compilation. 2024. MLC LLM. https://llm.mlc.ai/

[7] Georgi Gerganov. 2024. llama.cpp. https://github.com/ggerganov/llama.cpp

[8] HuggingFace. 2024. Large Language Model Text Generation Inference. https://huggingface.co/docs/text-generation-inference/index

[9] Intel. 2022. Intel® Performance Counter Monitor - A Better Way to Measure CPU Utilization. https://www.intel.com/content/www/us/en/developer/articles/tool/performance-counter-monitor.html

[10] Intel. 2024. Intel® Advanced Vector Extensions. https://software.intel.com/en-us/isa-extensions/intel-avx

[11] Intel. 2024. Intel® oneAPI Math Kernel Library (oneMKL). https://www.intel.com/content/www/us/en/developer/tools/oneapi/onemkl.html

[12] Jinwoo Jeong, Seungsu Baek, and Jeongseob Ahn. 2023. Fast and Efficient Model Serving Using Multi-GPUs with Direct-Host-Access. In *Proceedings of the Eighteenth European Conference on Computer Systems* (Rome, Italy) *(EuroSys '23)*. Association for Computing Machinery, New York, NY, USA, 249–265. https://doi.org/10.1145/3552326.3567508

[13] Guillaume Klein, Dakun Zhang, Clément Chouteau, Josep Crego, and Jean Senellart. 2020. Efficient and High-Quality Neural Machine Translation with OpenNMT. In *Proceedings of the Fourth Workshop on Neural Generation and Translation*, Alexandra Birch, Andrew Finch, Hiroaki Hayashi, Kenneth Heafield, Marcin Junczys-Dowmunt, Ioannis Konstas, Xian Li, Graham Neubig, and Yusuke Oda (Eds.). Association for Computational Linguistics, Online, 211–217. https://doi.org/10.18653/v1/2020.ngt-1.25

[14] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. arXiv:cs.LG/2309.06180

[15] Jan Karel Lenstra, David B Shmoys, and Éva Tardos. 1990. Approximation algorithms for scheduling unrelated parallel machines. *Mathematical programming* 46 (1990), 259–271.

[16] Microsoft. 2024. DeepSpeed Model Implementations for Inference (MII). https://www.microsoft.com/en-us/research/project/deepspeed/deepspeed-mii/

[17] NVIDIA. 2024. NVIDIA A10 GPU Accelerator. https://www.nvidia.cn/content/dam/en-zz/Solutions/Data-Center/a10/pdf/A10-Product-Brief.pdf

[18] NVIDIA. 2024. NVIDIA A100 Tensor Core GPU. https://www.nvidia.cn/data-center/a100/

[19] OpenMP. 2024. OpenMP. https://www.openmp.org/

[20] PyTorch. 2024. PyTorch Docs. https://pytorch.org/

[21] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. 2019. Language models are unsupervised multitask learners. *OpenAI blog* 1, 8 (2019), 9.

[22] Bibhudatta Sahoo. 2013. *Dynamic load balancing strategies in heterogeneous distributed system*. Ph.D. Dissertation.

[23] Ray Serve. 2024. Ray Serve: Scalable and Programmable Serving. https://docs.ray.io/en/latest/serve/index.html

[24] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Beidi Chen, Percy Liang, Christopher Ré, Ion Stoica, and Ce Zhang. 2023. FlexGen: High-Throughput Generative Inference of Large Language Models with a Single GPU. In *Proceedings of the 40th International Conference on Machine Learning* (Honolulu, Hawaii, USA) *(ICML'23)*. JMLR.org, Article 1288, 23 pages.

[25] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. 2019. Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism. *CoRR* abs/1909.08053 (2019). arXiv:1909.08053 http://arxiv.org/abs/1909.08053

[26] Romal Thoppilan, Daniel De Freitas, Jamie Hall, Noam Shazeer, Apoorv Kulshreshtha, Heng-Tze Cheng, Alicia Jin, Taylor Bos, Leslie Baker, Yu Du, et al. 2022. Lamda: Language models for dialog applications. *arXiv preprint arXiv:2201.08239* (2022).

[27] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971* (2023).

[28] Hugo Touvron, Louis Martin, and Kevin R. Stone et al. 2023. Llama 2: Open Foundation and Fine-Tuned Chat Models. *ArXiv* abs/2307.09288 (2023). https://api.semanticscholar.org/CorpusID:259950998

[29] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is All You Need. In *Proceedings of the 31st International Conference on Neural Information Processing Systems* (Long Beach, California, USA) *(NIPS'17)*. Curran Associates Inc., Red Hook, NY, USA, 6000–6010.

[30] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. 2024. Tree of thoughts: Deliberate problem solving with large language models. *Advances in Neural Information Processing Systems* 36 (2024).

[31] Biao Zhang, Barry Haddow, and Alexandra Birch. 2023. Prompting large language model for machine translation: A case study. *arXiv preprint arXiv:2301.07069* (2023).

[32] Susan Zhang, Stephen Roller, Naman Goyal, Mikel Artetxe, Moya Chen, Shuohui Chen, Christopher Dewan, Mona Diab, Xian Li, Xi Victoria Lin, et al. 2022. Opt: Open pre-trained transformer language models. *arXiv preprint arXiv:2205.01068* (2022).

[33] Tianyi Zhang, Faisal Ladhak, Esin Durmus, Percy Liang, Kathleen McKeown, and Tatsunori B Hashimoto. 2023. Benchmarking large language models for news summarization. *arXiv preprint arXiv:2301.13848* (2023).

[34] Jianqiao Zheng, Sameera Ramasinghe, and Simon Lucey. 2021. Rethinking positional encoding. *arXiv preprint arXiv:2107.02561* (2021).